

MongoDB Database Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 2 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/database-technologies/mongodb-training-in-pune>

Course Overview

MongoDB is the dominant NoSQL document database in Pune product engineering — almost every Pune Node.js / MERN stack codebase runs on Mongo, and Pune startups (especially fintech / consumer-tech / e-commerce) frequently pick Mongo as their primary database for fast development and flexible schema. Archer Infotech's MongoDB training in Pune teaches the database as it is actually used in 2026 — MongoDB 8.0 (released October 2024) as the production default, document modelling with embedding vs referencing, the rich query language, the aggregation pipeline (Mongo's analytics engine), indexing including the 2024 vector-search additions, transactions (yes, MongoDB has multi-document ACID transactions since 4.0), Mongoose as the de-facto Node.js ODM, plus the production tail (replica sets, sharding overview, MongoDB Atlas managed cloud). Classroom in Kothrud, online live, and weekend batches available.

Why Learn MongoDB in 2026

MongoDB is among the most-deployed databases in Pune product engineering — Indeed Pune lists 800+ active openings that explicitly require MongoDB, plus a thousand more Node.js / MERN / backend roles where Mongo is the institutional default. The biggest employers running Mongo at scale are Persistent Systems, BMW TechWorks India, Mastercard Pune Tech Hub, Synechron, plus the entire Pune SaaS / fintech / e-commerce startup scene (Amagi, Fyllo, BharatPe Pune, Razorpay Pune, Pine Labs Pune, Drip Capital, Innovaccer Pune, Whatfix Pune). MongoDB Atlas (the managed-cloud variant) has become the de-facto choice for many startups — easier operations than self-hosting.

What changed in 2026: MongoDB 8.0 (October 2024) is the current production default — significant performance improvements (30%+ on common workloads), Queryable Encryption GA, plus better time-series and vector-search support. MongoDB Atlas Vector Search has matured into a credible alternative to pgvector / dedicated vector databases for AI / RAG use cases. Atlas Search (full-text search via Lucene) has matured. The Atlas-Stream-Processing service has emerged for stream-processing workflows. Plus the MongoDB ecosystem has tightened around Mongoose for Node.js + the official drivers for Python / Java / .NET.

What this means for hiring: 2026 Pune MongoDB JDs expect document-modelling fluency (embedding vs referencing, the discipline of denormalising for read patterns), aggregation-pipeline depth, indexing (including text and vector indexes), plus basic Atlas operations. Senior roles add replica sets, sharding, transactions, plus the Atlas-Search / Atlas-Vector-Search patterns.

- 800+ active Pune openings explicitly require MongoDB (May 2026)
- Plus another ~1,000 Node.js / MERN roles where Mongo is the default
- MongoDB 8.0 — current production default, 30%+ perf gains
- MongoDB Atlas — de-facto managed-cloud choice
- Atlas Vector Search — credible alternative to pgvector for RAG

Who This Course Is For

This course fits you if

- Working Node.js / MERN developer wanting MongoDB depth (the database your team probably runs)
- Working backend developer in another stack wanting to add MongoDB
- Engineering / BCS / MCA student preparing for our MERN Stack track or Node.js track
- Working Python / Java / .NET developer wanting MongoDB skill for full-stack roles
- Career restarter targeting Pune SaaS / fintech / e-commerce backend
- Working SQL DBA wanting to add NoSQL document database to your skill stack

This course is not the right fit if

- If you want enterprise relational DBA depth — pick Oracle or PostgreSQL
- If your goal is BFSI Capital Markets DBA premium — pick Oracle (BFSI runs more Oracle)
- If your goal is heavy-OLTP transactional workloads (financial-ledger style) — Postgres or Oracle is usually the right tool
- If you cannot put in 6–8 hours per week of practice outside class
- If you have 3+ years of production MongoDB experience — talk to us about advanced specialisations

Detailed Curriculum

Module 1: Document Databases & the MongoDB Server (Week 1)

What a document database is, and — more usefully — what problem it was built for. The document model set against the relational one, where the join-free read of a whole entity wins, where it loses, and why 'schemaless' is a description of the server rather than permission to skip designing a schema.

BSON is opened up properly: the binary format, its types beyond JSON — ObjectId, Date, Decimal128, binary and 64-bit integers — and the document size limit of 16MB that quietly shapes every modelling decision later in the course.

MongoDB 8.0 is installed locally, in Docker and on an Atlas free-tier cluster, and mongosh and Compass are set up alongside each other so a student can read a query in one and run it in the other.

- The document model against the relational model
- Where document databases win, and where they do not
- Why schemaless does not mean design-free
- BSON types — ObjectId, Date, Decimal128, binary, Int64
- The 16MB document limit and what it constrains
- MongoDB 8.0 locally, in Docker and on Atlas free tier
- mongosh, MongoDB Compass and the VS Code extension
- Databases, collections and namespaces

Module 2: CRUD, Query Operators & the Shell (Week 1)

Everyday read and write work, done fluently. insertOne and insertMany, find and findOne with projection, sort, skip and limit, and the full query-operator vocabulary — comparison, logical, element, evaluation and array operators — practised until a student stops reaching for the documentation for \$in, \$exists and \$elemMatch.

Updates are given proper time because they are where document databases differ most: \$set, \$unset, \$inc, \$push and \$pull, positional operators for updating one element inside an array, upserts, and aggregation-pipeline updates that compute a new value from the existing document. Deletes, bulk writes, cursors and findOneAndUpdate close the module.

- insertOne, insertMany and write results
- find and findOne with projection, sort, skip, limit
- Comparison, logical and element query operators
- Array queries — \$elemMatch, \$all, \$size
- \$set, \$unset, \$inc, \$push, \$pull and \$addToSet
- Positional operators and updating array elements
- Upserts and findOneAndUpdate
- Aggregation-pipeline updates
- Bulk writes and cursor behaviour

Module 3: Data Types, Schema Validation & Collections (Week 2)

Putting a schema back into a database that does not demand one. JSON Schema validation on a collection, validation levels and actions, and the practical rollout — start with warn on an existing collection, fix the documents that fail, then move to error — because switching validation on hard in production rejects writes nobody expected to fail.

Type choices then get the attention they deserve: Decimal128 for money, dates stored as Date rather than strings, ObjectId versus a natural key, and how a numeric string sneaking into an otherwise numeric field breaks both queries and indexes. Capped collections, TTL collections for expiring data, and native time-series collections close the module.

- JSON Schema validation on a collection
- Validation levels and actions, and a safe rollout
- Decimal128 for money; Date rather than date strings
- ObjectId against a natural key
- Type mismatches that silently break queries and indexes
- Capped collections
- TTL indexes for expiring documents
- Time-series collections
- Naming conventions and collection layout

Module 4: Document Modelling — Embedding vs Referencing (Week 2)

The decision that determines whether a MongoDB application is fast or is a support ticket, and the single topic most often examined in a MongoDB interview. Embedding against referencing, decided by read and write frequency, cardinality, whether the child is ever queried alone, and how the document grows over its lifetime.

The standard relationships are worked through with real briefs: one-to-few embedded, one-to-many by reference, and many-to-many. Unbounded array growth is demonstrated as the classic failure — an array that grows for the life of the parent will eventually exceed 16MB, and long before that it destroys write performance. \$lookup is introduced honestly as a real join that is nonetheless not a reason to model relationally in a document store.

- Embedding against referencing — the deciding questions
- Read frequency, write frequency and cardinality
- One-to-few, one-to-many and many-to-many
- Unbounded array growth as the classic failure
- Document growth over its lifetime
- Denormalising fields that are read together
- Keeping duplicated data consistent
- \$lookup, and what it costs
- Modelling from access patterns, not from entities

Module 5: Modelling Patterns for Real Workloads (Week 3)

The named patterns experienced MongoDB engineers reach for, each introduced by the problem it solves. The bucket pattern for high-frequency readings; the computed pattern for values that would otherwise be aggregated on every read; the extended reference for the two or three fields you always need from a related document; the attribute pattern for sparse and varied fields; the outlier pattern for the one document in a million that breaks the model.

Schema versioning is treated as a first-class requirement: a schemaVersion field, application code that reads more than one version, and a background migration that moves documents forward without downtime. Polymorphic collections, hierarchies with materialised paths, and audit and history documents complete the module, and each pattern is applied to a Pune-realistic brief rather than described abstractly.

- The bucket pattern for high-frequency data
- The computed pattern and precomputed aggregates
- The extended reference pattern
- The attribute pattern for sparse fields
- The outlier pattern
- Schema versioning and rolling migrations

- Polymorphic collections and discriminators
- Hierarchies — materialised paths and ancestor arrays
- Audit trails and document history

Module 6: The Aggregation Framework — Core Stages (Week 3)

MongoDB's analytics engine, and the feature that most self-taught candidates underuse. The pipeline as a sequence of transformations, then the core stages in the order they are usually needed: `$match` first and as early as possible, `$project` and `$addFields` to shape, `$group` with its accumulators, `$sort`, `$limit` and `$skip`, and `$unwind` for flattening arrays.

Expression operators are then built up — arithmetic, string, date, conditional `$cond` and `$switch`, and the array operators `$map`, `$filter` and `$reduce` — so that a pipeline can compute rather than merely filter. Every pipeline is written against a dataset of over ten million documents, which makes the difference between `$match` before and after `$unwind` something a student measures rather than takes on trust.

- The pipeline model and stage order
- `$match` early, and why order decides cost
- `$project`, `$addFields` and `$set` for shaping
- `$group` and the accumulator operators
- `$sort`, `$limit`, `$skip` and sort memory limits
- `$unwind` and flattening arrays
- Arithmetic, string and date expression operators
- `$cond` and `$switch` for conditional logic
- `$map`, `$filter` and `$reduce` over arrays

Module 7: Advanced Aggregation & Analytics (Week 4)

The stages that turn the pipeline into a reporting engine. `$lookup` for joins including the correlated sub-pipeline form, `$graphLookup` for recursive traversal of hierarchies, `$facet` for running several pipelines over one input in a single pass — the stage behind almost every search page with counts beside its filters — and `$bucket` and `$bucketAuto` for histograms.

`$setWindowFields` brings SQL-style window functions to MongoDB: running totals, rankings and moving averages without a self-join. `$merge` and `$out` materialise results into a collection for scheduled reporting, and the module ends on performance — the `allowDiskUse` flag, the 100MB per-stage memory limit, and reading an aggregation explain plan to find the stage that is doing the damage.

- `$lookup`, including correlated sub-pipelines
- `$graphLookup` for recursive hierarchies
- `$facet` for multi-pipeline single-pass reporting
- `$bucket` and `$bucketAuto` for histograms
- `$setWindowFields` — ranking, running totals, moving averages
- `$merge` and `$out` for materialised results
- `$unionWith` across collections
- Memory limits and `allowDiskUse`
- Reading an aggregation explain plan

Module 8: Indexing, Query Planning & Performance Tuning (Weeks 4–5)

Why one query returns instantly and another scans the collection. Index types in full — single-field, compound, multikey over arrays, text, wildcard, hashed, 2dsphere for geospatial, and partial and sparse indexes that index only the documents anyone queries — with the ESR rule (equality, sort, range) as the principle that decides compound-index field order.

Diagnosis follows. `explain('executionStats')` read properly, IXSCAN against COLLSCAN, the ratio of documents examined to documents returned as the number that tells the truth, and covered queries that never touch a document at all. The database profiler and Atlas Performance Advisor point at what to fix first, and the module closes on the anti-patterns that cause most real slowdowns: unbounded arrays, case-insensitive regex, `$ne` and `$nin` on a large collection, and deep skip pagination.

- Single-field, compound and multikey indexes
- Text, wildcard, hashed and 2dsphere indexes
- Partial and sparse indexes
- The ESR rule for compound index order
- explain('executionStats') and the winning plan
- IXSCAN vs COLLSCAN and examined-to-returned ratio
- Covered queries and index-only reads
- The database profiler and Atlas Performance Advisor
- Anti-patterns — regex, \$ne, deep skip, unbounded arrays

Module 9: Transactions, Write Concerns & Consistency (Week 5)

Correctness guarantees, and the interview question that MongoDB has no transactions — which has been wrong since version 4.0. Single-document atomicity first, because a well-modelled document usually removes the need for a transaction; then multi-document ACID transactions, their session and retry semantics, and their real cost, so that a student can argue when one is warranted.

Write concern and read concern are then taught as the dials that actually decide durability and visibility: `w: 1` against `w: majority`, `journaling`, `readConcern: majority` and `linearizable`, read preference across a replica set, causal consistency, and change streams as the supported way to react to writes. Retryable writes and idempotent handlers close the module.

- Single-document atomicity, and modelling to use it
- Multi-document ACID transactions and sessions
- When a transaction is worth its cost
- Write concern — `w:1`, `w:majority` and `journaling`
- Read concern — `local`, `majority`, `linearizable`
- Read preference and reading from secondaries
- Causal consistency and session guarantees
- Change streams and reacting to writes
- Retryable writes and idempotent handlers

Module 10: Application Integration — Drivers, Mongoose & Spring Data (Week 6)

Connecting MongoDB to the code that uses it, across the stacks Pune teams actually run. The connection string and its options, the driver's built-in connection pool, timeouts and retry behaviour, and how a serverless deployment exhausts a pool that a long-running server never would.

Mongoose is covered at working depth for Node.js — schemas and validation, pre- and post-hooks, `virtuals`, `populate` and its N+1 cost, `lean` queries and discriminators — alongside the honest question of when the raw driver is the better choice. Spring Data MongoDB and Python's PyMongo and Motor are covered for the Java and Python stacks. Credentials move into environment variables, and the whole course's work is kept in Git with branches, a README and a documented data model.

- Connection strings, pooling, timeouts and retries
- Mongoose schemas, validation and middleware
- `Virtuals`, `populate` and its N+1 cost
- `lean` queries and discriminators
- Mongoose against the raw driver
- Spring Data MongoDB for the Java stack
- PyMongo and `async Motor` for Python
- Credentials in environment variables and secret stores
- Git, README and a documented data model

Module 11: Atlas Search & Atlas Vector Search (Week 6)

Search that does not need a second system. Atlas Search is Lucene running beside the database: analysers and tokenisers, the `$search` stage, compound queries with `must`, `should` and `filter`, fuzzy matching for misspelled input,

autocomplete, faceting, and relevance scoring you can inspect and tune.

Atlas Vector Search follows, and it is why several Pune AI features never leave MongoDB: embedding fields, approximate nearest-neighbour search, the number of candidates against recall, and pre-filtering so a similarity search still respects tenant and permission boundaries. A retrieval-augmented generation flow is built end to end — embed, store, retrieve, rank — and compared honestly with the PostgreSQL and pgvector alternative.

- Atlas Search indexes, analysers and tokenisers
- The \$search stage and compound queries
- Fuzzy matching, autocomplete and synonyms
- Faceting and relevance scoring
- Vector Search — embeddings and ANN indexes
- numCandidates, limit and the recall trade-off
- Pre-filtering for tenant and permission boundaries
- A retrieval-augmented generation flow, end to end
- Atlas Vector Search against Postgres with pgvector

Module 12: Replica Sets & High Availability (Week 7)

Running MongoDB so that losing a machine is an event rather than an outage. A three-node replica set is built by hand in class — primary, secondaries and the oplog — then an election is triggered by killing the primary and watched from the driver's side, which is the part that makes the abstraction real.

The oplog is examined as the mechanism behind both replication and change streams: its window, replication lag and the stale reads a secondary read preference can produce, and initial sync. Arbiters and why they are usually a mistake, priority and hidden and delayed members, rolling maintenance without downtime, and how the driver discovers topology and retries during failover complete the module.

- Building a three-node replica set by hand
- Primary, secondaries and the election process
- The oplog, its window and initial sync
- Replication lag and stale secondary reads
- Arbiters, and why they are usually a mistake
- Priority, hidden and delayed members
- Rolling maintenance without downtime
- Driver topology discovery and retries during failover
- Monitoring replication health

Module 13: Sharding & Scaling Out (Week 7)

Horizontal scale, and the honest advice that most Pune workloads should not shard yet. When sharding is genuinely warranted, what it costs operationally, and the vertical and read-scaling options that should be exhausted first.

Then the mechanism: shard key selection as the decision everything else follows from, ranged against hashed keys, cardinality, frequency and monotonicity — a timestamp or an incrementing id concentrates every write on one shard — the config servers and mongos router, chunks and the balancer, targeted queries against scatter-gather, and zone sharding for data residency, which matters under India's DPDP Act. Resharding an already-sharded collection closes the module.

- When to shard, and what to try first
- Shard key selection and its consequences
- Ranged against hashed shard keys
- Cardinality, frequency and monotonic keys
- Config servers, mongos and the balancer
- Chunks, splits and migrations
- Targeted queries against scatter-gather
- Zone sharding for data residency
- Resharding an existing collection

Module 14: Security, Atlas Operations & Backup (Week 8)

Everything that stands between a working cluster and a breach notification. Authentication with SCRAM and x.509, role-based access control with built-in and custom roles, least privilege applied to an application user, network isolation through IP access lists and VPC peering, and TLS everywhere — introduced against the well-documented history of unauthenticated MongoDB instances left open to the internet.

Encryption follows: at rest, in transit, and Queryable Encryption for fields that must stay encrypted while remaining searchable. Atlas operations then cover cluster tiers and autoscaling, alerts, the Performance Advisor, cost control, continuous backup and a genuine point-in-time restore performed by every student, plus mongodump and mongorestore and the migration path from a self-hosted deployment.

- SCRAM and x.509 authentication
- Built-in and custom roles, and least privilege
- IP access lists, VPC peering and private endpoints
- TLS in transit and encryption at rest
- Queryable Encryption for searchable encrypted fields
- Atlas tiers, autoscaling, alerts and cost control
- Continuous backup and a verified point-in-time restore
- mongodump, mongorestore and migration from self-hosted
- Auditing and DPDP obligations

Module 15: Capstone Project & Interview Preparation (Week 9 + placement prep)

The capstone runs the whole course through one system: a data model designed from a written brief and defended against its access patterns, loaded with millions of documents, reported on through aggregation pipelines, indexed against measured explain output, made searchable with Atlas Search and a vector-search feature, deployed on a replica set with a tested restore, and documented well enough that a stranger can run it.

Interview preparation targets what Pune panels actually ask a MongoDB candidate: justify an embedding decision, write an aggregation pipeline live, explain the ESR rule, describe what majority guarantees, and say when you would not use MongoDB at all. Resume, LinkedIn and GitHub are rewritten around the capstone, and two mock interviews are run and reviewed.

- Capstone data model, defended against access patterns
- Data load, aggregation reporting and dashboards
- A measured before-and-after indexing exercise
- An Atlas Search and vector-search feature
- Replica-set deployment and a verified restore
- README, data-model diagram and API documentation
- Live pipeline writing under interview conditions
- Resume, LinkedIn and GitHub rewritten around the capstone
- Two mock interviews with recorded feedback

Module 16: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI
- AI debugging — interpreting errors, logs, and failing output

- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

MERN-Style E-Commerce Backend with Aggregation Analytics

A complete MERN-style backend on MongoDB — pick a real domain (e-commerce, fintech transactions, ticketing, inventory). Document modelling with proper embed-vs-reference decisions, 10M+ documents synthetic data, 20+ aggregation-pipeline analytics queries (top sellers, cohort retention, RFM analysis), proper indexing with explain() output. Optionally a small Express + Mongoose service on top.

- Technology: MongoDB 8.0 + Atlas
- Technology: Aggregation pipeline depth
- Technology: Indexing + explain()
- Technology: Mongoose ODM
- Technology: Express service (optional)
- Technology: 10M+ document dataset

Atlas Vector Search RAG Service

A retrieval-augmented generation service backed by MongoDB Atlas Vector Search — pick a real domain corpus, embed via OpenAI, store in Atlas Vector Search, hybrid retrieval (Atlas Search BM25 + Vector Search dense + reranking), plus a small FastAPI / Express endpoint. Demonstrates the 2026 Mongo differentiator alongside pgvector.

- Technology: MongoDB Atlas Vector Search
- Technology: OpenAI embeddings
- Technology: Atlas Search (Lucene)
- Technology: FastAPI or Express endpoint

Time-Series + Real-Time App

A time-series application using MongoDB time-series collections (IoT sensor data, app events, financial tick data) with continuous queries, retention policies, plus a small real-time dashboard.

- Technology: MongoDB 8.0 time-series collections
- Technology: Aggregation pipeline for analytics
- Technology: Change Streams for real-time
- Technology: Streamlit or simple frontend

Career Outcomes

MongoDB fluency is a near-mandatory expectation on Pune Node.js / MERN / backend roles — Indeed Pune lists 800+ openings explicitly requiring MongoDB, plus another ~1,000 Node.js / MERN roles where it is the default. The biggest employers are Persistent Systems, BMW TechWorks India, Mastercard Pune Tech Hub, Synechron, plus the entire Pune SaaS / fintech / e-commerce startup scene.

What pulls a MongoDB developer above the median band: depth on document modelling (embed-vs-reference decisions), demonstrable aggregation-pipeline work, indexing literacy with explain() output, plus one Atlas Vector Search project (the 2026 differentiator). Our capstone projects are designed exactly around these signals.

Senior MongoDB Developer / DBA bands at the top end are reported as national figures (Pune-specific Indeed pages do not exist for these specific titles); Pune trends within $\pm 10\%$ of these figures.

Roles after this course

- MongoDB Developer
- Backend Developer (MongoDB-heavy)
- Full Stack Developer (MERN)

- Junior MongoDB DBA
- AI Engineer (with Atlas Vector Search)

Indicative salary bands

- MongoDB Developer (Pune) - Rs. 6,52,000 per year average (source: Indeed Pune (MongoDB Developer))
- Junior MongoDB Developer (Pune entry, <2 years) - Rs. 3,80,000 – Rs. 6,80,000 per year (source: AmbitionBox Pune MongoDB Developer)
- Mid-level MongoDB Developer (Pune, 3–5 years) - Rs. 10,00,000 – Rs. 16,00,000 per year (source: Glassdoor Pune MongoDB Developer)
- Senior MongoDB DBA / Engineer (Pune, 5–8 years) - Rs. 15,00,000 – Rs. 26,00,000 per year (source: Glassdoor Pune Senior MongoDB DBA)

Companies hiring for these skills

- Persistent Systems
- BMW TechWorks India
- Mastercard Pune Tech Hub
- Synechron
- Bajaj Finserv (some teams)
- Amagi
- Fyllo
- BharatPe (Pune)
- Razorpay (Pune)
- Pine Labs (Pune)
- Drip Capital
- Innovaccer (Pune)
- Whatfix (Pune)
- TCS
- Cognizant
- Capgemini

Batch Modes and Duration

Duration: 8 weeks of structured curriculum plus 1 week of capstone (~2 months total)

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- MongoDB Atlas free tier per student
- MongoDB Compass + mongosh
- GitHub for capstone
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~3.5 months instead of 2.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession. MongoDB Atlas free tier covers lab work for most students.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2 instalments at no extra cost
- Corporate sponsorship — invoiced with GST

Placement Support

Placement support starts from week 6. By the end of the curriculum your resume highlights real document-modelling and aggregation-pipeline work, your GitHub has at least two production-style repositories, and you have completed at least two mock technical interviews.

We say placement support, not placement guarantee. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Week 6 — resume + LinkedIn rewrite
- Week 7 — GitHub portfolio cleanup
- Weeks 8–9 — two rounds of mock technical interviews
- Week 9 — HR mock and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network at 12+ partner companies (with extra emphasis on Pune SaaS / fintech / MERN stacks)
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- Persistent Systems
- BMW TechWorks India
- Synechron
- Amagi
- BharatPe (Pune)
- Razorpay (Pune)
- Pine Labs (Pune)
- Drip Capital
- Innovaccer (Pune)
- TCS
- Cognizant

Prerequisites and How to Start

Prerequisites: basic computer use, familiarity with JSON (the core skill), basic JavaScript / Python helps but isn't required. No prior database experience required. Willingness to commit 6–8 hours per week of practice.

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment and complete pre-course orientation (MongoDB Atlas free-tier signup, mongosh + Compass install)
- Show up to day one with a laptop running 64-bit OS

Frequently Asked Questions

How long does MongoDB training in Pune take at Archer Infotech?

Approximately 2 months — 8 weeks plus 1 week of capstone. Weekend batch ~3.5 months.

MongoDB or PostgreSQL?

Mongo for Pune Node.js / MERN / SaaS / fintech / consumer-tech. Postgres for relational + transactional + analytics. Both well-paid with ample Pune hiring.

What is the salary of a MongoDB Developer in Pune?

Indeed Pune Rs. 6.52 lakh average. Junior Rs. 3.8–6.8 lakh per AmbitionBox. Mid-level Rs. 10–16 lakh per Glassdoor. Senior Rs. 15–26 lakh.

Does MongoDB have transactions?

Yes — multi-document ACID transactions since MongoDB 4.0. Common misconception is they don't. We cover when to use transactions vs when atomic single-document updates suffice.

Will I work on real projects?

Yes — three capstone projects: (1) MERN-style e-commerce with aggregation analytics on 10M+ documents, (2) Atlas Vector Search RAG service, (3) time-series + real-time app.

Is Atlas Vector Search / RAG covered?

Yes — week 6 covers Atlas Vector Search and the RAG patterns. Capstone Project #2 is a complete Atlas Vector Search RAG service.

Are weekend MongoDB classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~3.5 months.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network (extra emphasis on Pune SaaS / fintech / MERN stacks), mock interviews.

Are the named trainers actually teaching?

Amol Patil and Ankita Hartale personally lead every session.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.